

Luan Italo Mota Sousa

Análise de Sistemas Discretos

Maracanaú - CE
21 de abril de 2026

Luan Italo Mota Sousa

Análise de Sistemas Discretos

Relatório apresentado à disciplina de Controle 2 como requisito avaliativo.

Instituto Federal de Educação, Ciência e Tecnologia do Ceará
Campus Maracanaú

Maracanaú - CE
21 de abril de 2026

Sumário

Sumário	2
0.1 Introdução	3
0.2 Fundamentação Teórica	3
0.2.1 Invariância ao Impulso	3
0.2.2 ZOH (Retenção de Ordem Zero)	3
0.3 Metodologia	4
0.3.1 MATLAB	4
0.3.2 ESP32	5
0.4 Resultados	8
0.4.1 Simulações no MATLAB	8
0.4.2 Implementação no ESP32	12
0.5 Conclusão	15
REFERÊNCIAS	17

0.1 Introdução

Este trabalho tem como objetivo analisar e comparar diferentes métodos de discretização de sistemas no contexto de Controle Digital. Para isso, foram selecionadas quatro funções de transferência extraídas da Tabela 3.1 do livro Controle de Sistemas Amostrados, especificamente os itens 4, 8, 11 e 12.

A tabela já apresenta a função $m(nT)$ e sua respectiva transformada $M(z) = \mathcal{Z}\{m(nT)\}$, obtidas por meio do método da invariância ao impulso. Embora este método represente bem o sistema para entrada impulso, ele pode não apresentar bons resultados para entradas como degrau e rampa. Por isso, também foi utilizado o método de retenção de ordem zero (ZOH).

A análise foi realizada por meio de simulações no MATLAB e, posteriormente, implementada em uma plataforma ESP32, permitindo a comparação entre os resultados teóricos e práticos.

0.2 Fundamentação Teórica

A discretização de sistemas contínuos é um passo fundamental no projeto de sistemas de controle digital, permitindo que plantas descritas no domínio do tempo sejam representadas no domínio discreto (z). Os Dois métodos utilizados para essa finalidade foi o Invariância ao Impulso e o equivalente de Retenção de Ordem Zero (ZOH).

0.2.1 Invariância ao Impulso

O método de invariância da resposta ao impulso visa obter uma representação em tempo discreto $M(z)$ cuja sequência de saída corresponda exatamente às amostras da resposta ao impulso do sistema contínuo original $M(s)$ nos instantes de amostragem. No entanto, neste trabalho, não foi necessário realizar esse cálculo, pois a tabela já fornece diretamente $M(z)$, sendo necessária apenas a sua implementação.

0.2.2 ZOH (Retenção de Ordem Zero)

O método do Equivalente de Retenção de Ordem Zero (ZOH - *Zero-Order Hold*), também conhecido como "invariância ao degrau", é o modelo mais fiel para representar sistemas de controle reais onde existe um conversor digital-analógico entre o computador e a planta. Diferente da invariância ao impulso, este método considera o efeito físico do segurador, que mantém o valor do sinal constante durante todo o intervalo de amostragem T . A função de transferência de um ZOH no domínio de Laplace é dada por:

$$H_z(s) = \frac{1 - e^{-Ts}}{s}$$

Para obter o equivalente discreto $M(z)$ de uma planta $M(s)$ sob a ação de um ZOH, utiliza-se a seguinte formulação:

$$M(z) = \frac{z-1}{z} \mathcal{Z} \left\{ \frac{M(s)}{s} \right\}$$

Este método garante que a resposta do sistema discreto a um degrau unitário coincida exatamente com a resposta do sistema contínuo original nos instantes de amostragem.

0.3 Metodologia

Para os casos analisados, foi utilizado o valor $a = 1,5$ para as funções que dependem desse parâmetro, bem como um período de amostragem $T = 0,1$.

0.3.1 MATLAB

No MATLAB, inicialmente foi criada a representação em tempo contínuo utilizando $M(s)$ da Tabela 2 do livro citado, na qual são fornecidas a função $m(t)$ e sua transformada $M(s) = \mathcal{L}\{m(t)\}$. Para isso, foi utilizado o comando:

```
sys_c = tf(num, den)
```

Em seguida, foi aplicada a discretização pelo método de retenção de ordem zero (ZOH) por meio do comando:

```
sys_zoh = c2d(sys_c, T, 'zoh')
```

Para a discretização utilizando o método de invariância ao impulso, foram utilizados diretamente os valores fornecidos na Tabela 3, onde já se tem $M(z) = \mathcal{Z}\{m(nT)\}$. Nesse caso, o sistema discreto foi implementado com:

```
sys_imp = tf(num_inv, den_inv, T)
```

Por fim, foram aplicadas entradas do tipo impulso, degrau e rampa, permitindo a análise do comportamento dos sistemas discretizados.

```
[y_s_im, t_s_out_im] = impulse(sys_c, t_final);  
y_z_imp = lsim(sys_zoh, u_imp, t_z);  
y_Z_im_2 = lsim(sys_imp, u_imp, t_z);
```

0.3.2 ESP32

Para a implementação na ESP32, foi necessário calcular $M(z)$ para, em seguida, obter a equação de diferenças correspondente a cada sistema.

Como exemplo, para o item 4:

$$M(z) = \frac{z}{z-1} \mathcal{Z} \left\{ \frac{M(s)}{s} \right\}$$

Considerando $M(s) = \frac{1}{s^3}$, obtém-se:

$$M(z) = \frac{z}{z-1} \mathcal{Z} \left\{ \frac{1}{s^4} \right\}$$

Cálculo da Transformada Z de $\frac{1}{s^4}$:

$$\mathcal{Z} \left\{ \frac{1}{s^4} \right\} = \frac{T^3 z(z^2 + 4z + 1)}{6(z-1)^4}$$

Obtenção da Função de Transferência de Pulsos $M(z)$:

$$M(z) = \frac{z}{z-1} \cdot \frac{T^3 z(z^2 + 4z + 1)}{6(z-1)^4} = \frac{T^3(z^2 + 4z + 1)}{6(z-1)^3}$$

Expandindo:

$$M(z) = \frac{z^3 - 3z^2 + 3z - 1}{\frac{6}{T^3}z^2 + \frac{6}{4T^3}z + \frac{6}{T^3}}$$

Dedução da Equação de Diferenças:

$$\frac{U(z)}{Y(z)} = \frac{\frac{T^3}{6}z^{-1} + \frac{4T^3}{6}4T^3z^{-2} + \frac{T^3}{6}z^{-3}}{1 - 3z^{-1} + 3z^{-2} - z^{-3}}$$

Equação de Diferenças Final:

$$y(k) = 3y(k-1) - 3y(k-2) + y(k-3) + \frac{T^3}{6}u(k-1) + \frac{4T^3}{6}u(k-2) + \frac{T^3}{6}u(k-3)$$

No código ficou:

```
// =====  
// numerador  
// =====  
num[0] = 0;  
num[1] = (T * T * T) / 6;  
num[2] = (4 * (T * T * T)) / 6;  
num[3] = (T * T * T) / 6;
```

```

// =====
// denominador
// =====
den[0] = -1;
den[1] = 3;
den[2] = -3;
den[3] = 1;
y[0] = num[2] * u[2] + num[1] * u[1] + num[0] * u[0] - den[1] * y[1] ...
- den[2] * y[2] - den[3] * y[3];

```

Esses passos foram repetidos para o outro modelo, com uma diferença, não foi necessário obter $M(z)$, pois ele já estava disponibilizado na tabela. Assim, foi preciso apenas realizar a dedução da equação de diferenças. Dessa forma, os demais itens são apresentados na tabela a seguir.

Item	$M(z)$	Equação de Diferenças
4	$\frac{T^2 z(z+1)}{2(z-1)^3}$	$y(z) = \left(\frac{T^2}{2}\right)u(z-1) + \left(-\frac{T^2}{2}\right)u(z-2) - (-3)y(z-1) - 3y(z-2) - y(z-3)$
104	$\frac{T^3(z^2+4z+1)}{6(z-1)^3}$	$y(z) = \left(\frac{T^3}{6}\right)u(z-1) + \left(\frac{4T^3}{6}\right)u(z-2) + \left(\frac{T^3}{6}\right)u(z-3) - (-3)y(z-1) - 3y(z-2) - y(z-3)$
8	$\frac{z(1-e^{-aT})}{(z-1)(z-e^{-aT})}$	$y(z) = (1-e^{-aT})y(z-1) - (1-e^{-aT})y(z-1) - e^{-aT}y(z-2)$
108	$M(z) = \frac{\left(\frac{aT-1+e^{-aT}}{a}\right)z + \left(\frac{1-e^{-aT}-aTe^{-aT}}{a}\right)}{z^2 - (1+e^{-aT})z + e^{-aT}}$	$y(z) = \left(\frac{aT-1+e^{-aT}}{a}\right)u(z-1) + \left(\frac{aT-1+e^{-aT}}{a}\right)u(z) - (1-e^{-aT})y(z-1) - e^{-aT}y(z-2)$
11	$\frac{z \sin(aT)}{z^2 - 2 \cos(aT) z + 1}$	$y(z) = \text{sen}(aT)u(z-1) - (2\cos(aT))y(z-1) - zy(z-2)$
111	$\frac{(z - \cos(aT))(Z+1)}{a(z^2 - 2 \cos(aT) z + 1)}$	$y(z) = (1 - \cos(aT))u(z-1) - (2\cos(aT))u(z-2) - (2\cos(aT))y(z-1) - ay(z-2)$
12	$\frac{z(z - \cos(aT))}{z^2 - 2 \cos(aT) z + 1}$	$y(z) = u(z) - \cos(aT)u(z-1) - (2\cos(aT))y(z-1) - y(z-2)$
112	$\frac{(z-1) \sin(aT)}{a(z^2 - (2 \cos(aT))z + 1)}$	$y(z) = -\frac{2\cos(aT)}{a}u(z-1) - \frac{\text{sen}(aT)}{a}u(z-2) - 2\cos(aT)y(z-1) - y(z-2)$

Para diferenciar os métodos, foram utilizados os valores padrão para o invariante ao impulso. Para representar o ZOH, o item foi multiplicado por 100.

0.4 Resultados

0.4.1 Simulações no MATLAB

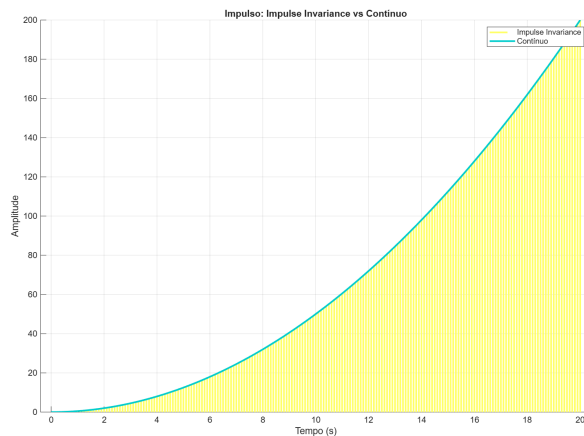
As simulações foram realizadas no ambiente MATLAB Online para validar o comportamento das funções de transferência discretizadas. Foram aplicados sinais de entrada do tipo impulso, degrau e rampa, permitindo observar a convergência dos métodos de Invariância ao Impulso e ZOH em relação ao modelo contínuo original.

A Figura 1 apresenta as curvas obtidas, onde se observa que o método de invariância ao impulso, conforme mencionado anteriormente, oferece maior precisão para este tipo de sinal de entrada. Em contrapartida, o método ZOH manteve as características da função de transferência, porém não foi capaz de representar adequadamente o comportamento do sistema neste cenário.

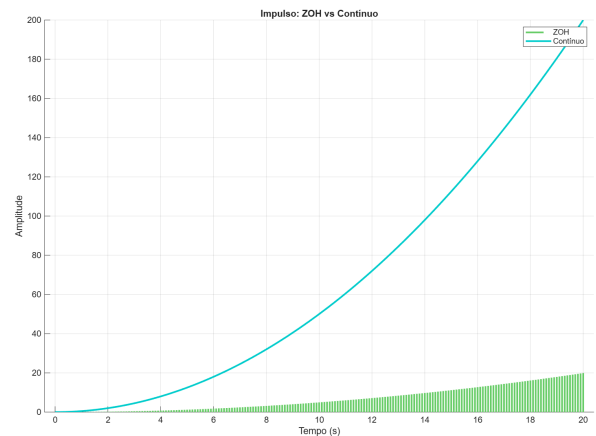
Na Figura 2, a situação muda o método ZOH agora é mais preciso que o outro. Isso acontece por causa da forma como esse método funciona, sendo melhor para lidar com entradas do tipo degrau.

Para o cenário em rampa, a Figura 3 mostra que a discretização ZOH continua com uma boa rastreabilidade quando comparada ao método invariante ao impulso.

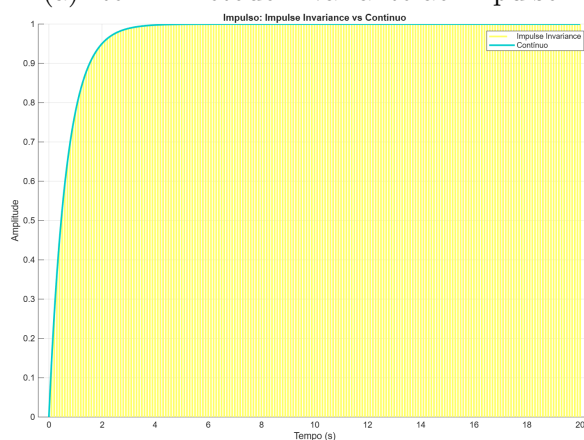
O código utilizado no MATLAB estará disponível no GitHub.



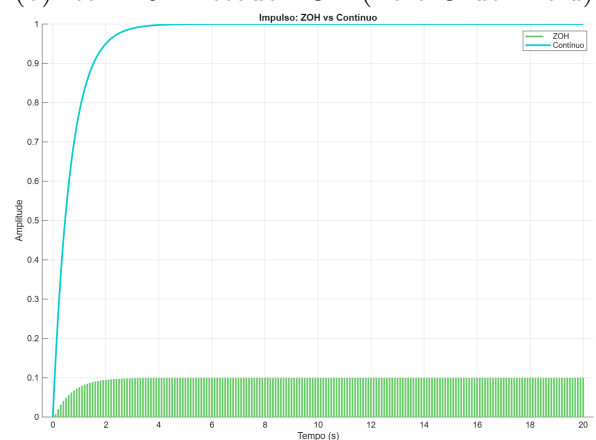
(a) Item 4:Método invariante ao impulso



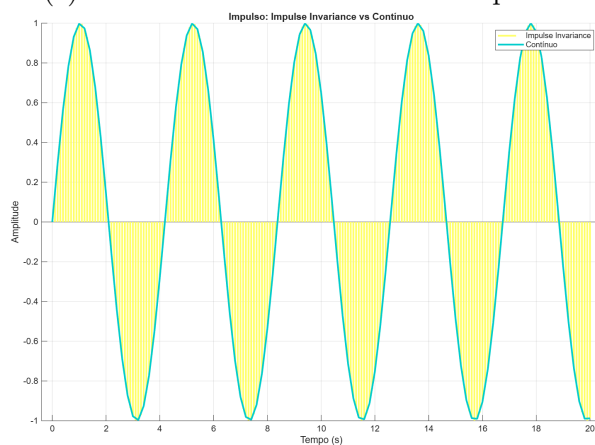
(b) Item 104:Método ZOH (Zero-Order Hold)



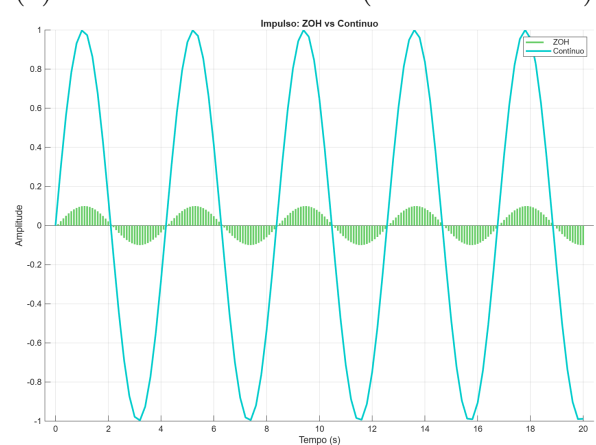
(c) Item 8:Método invariante ao impulso



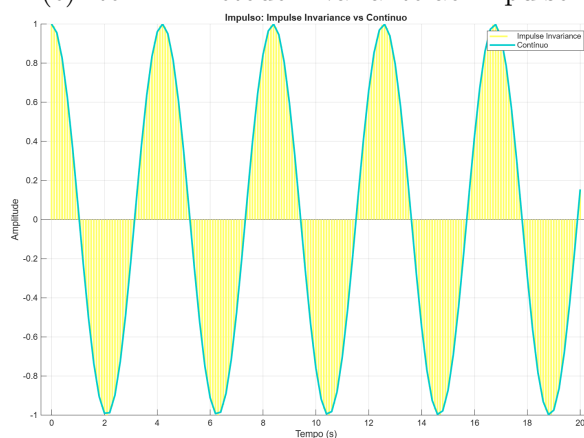
(d) Item 108:Método ZOH (Zero-Order Hold)



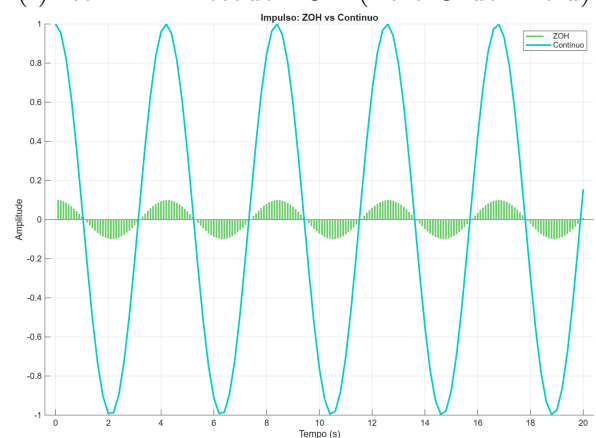
(e) Item 11:Método invariante ao impulso



(f) Item 111:Método ZOH (Zero-Order Hold)

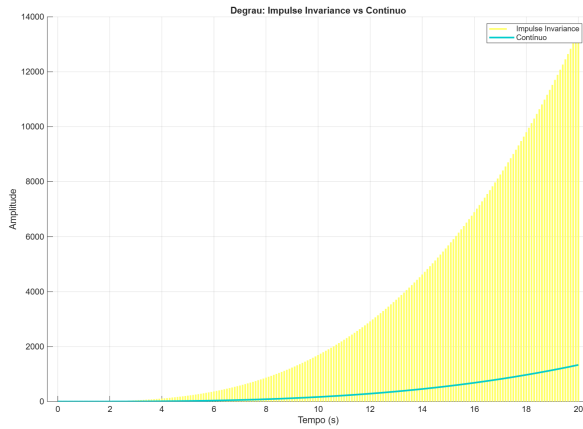


(g) Item 12: Método invariante ao impulso

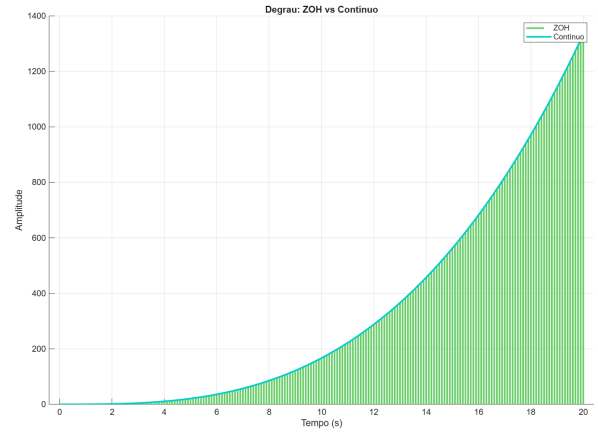


(h) Item 112:Método ZOH (Zero-Order Hold)

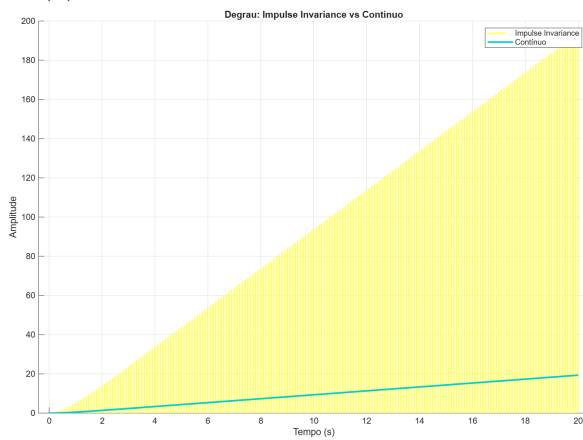
Figura 1 – Comparação das respostas ao impulso pelos dois métodos.



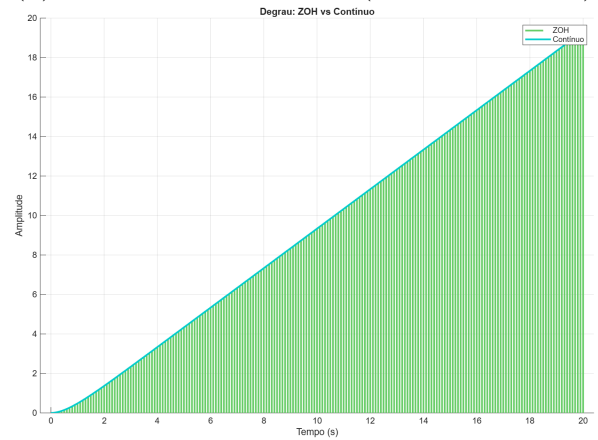
(a) Item 4:Método invariante ao impulso



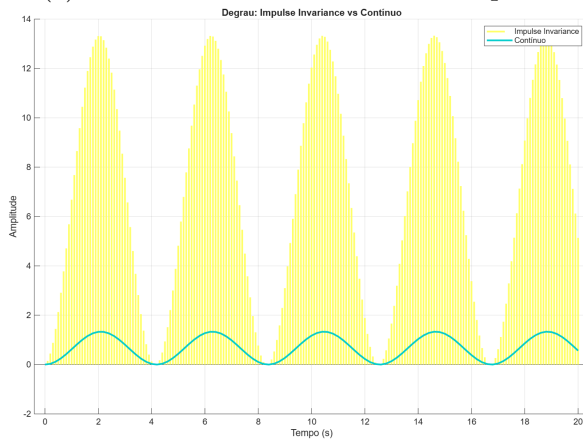
(b) Item 104:Método ZOH (Zero-Order Hold)



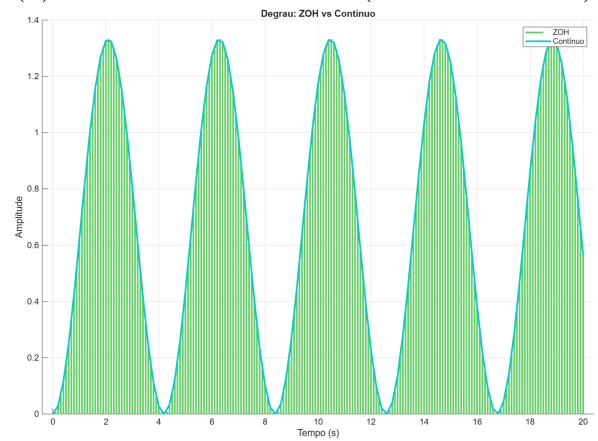
(c) Item 8:Método invariante ao impulso



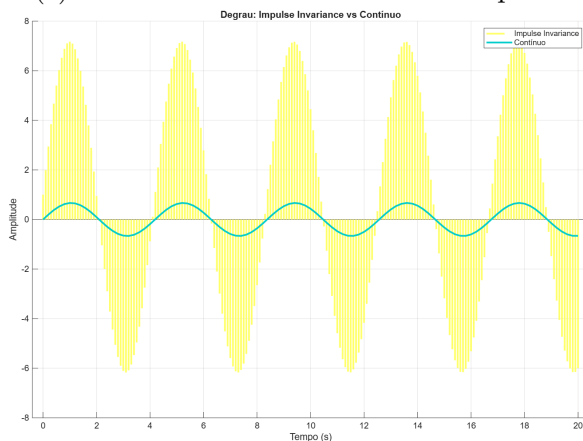
(d) Item 108:Método ZOH (Zero-Order Hold)



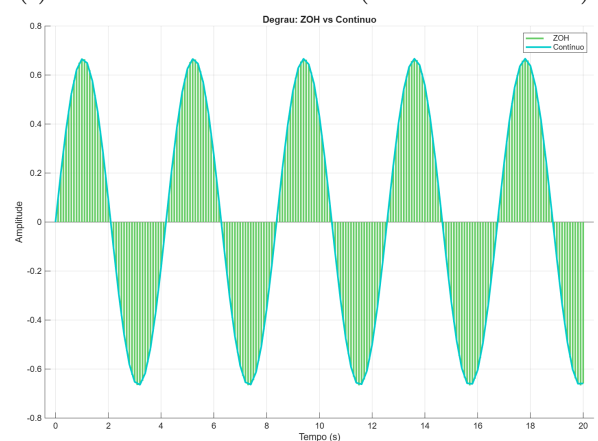
(e) Item 11:Método invariante ao impulso



(f) Item 111:Método ZOH (Zero-Order Hold)

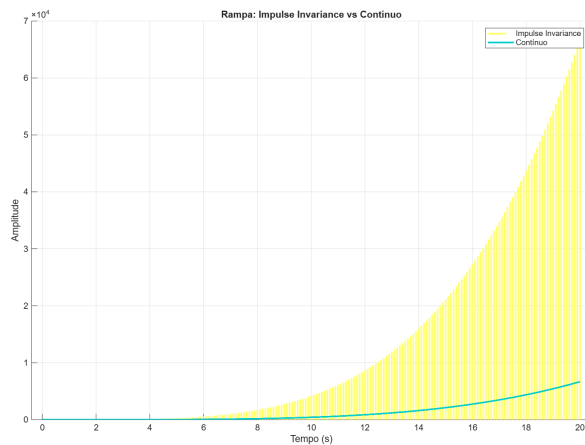


(g) Item 12:Método invariante ao impulso

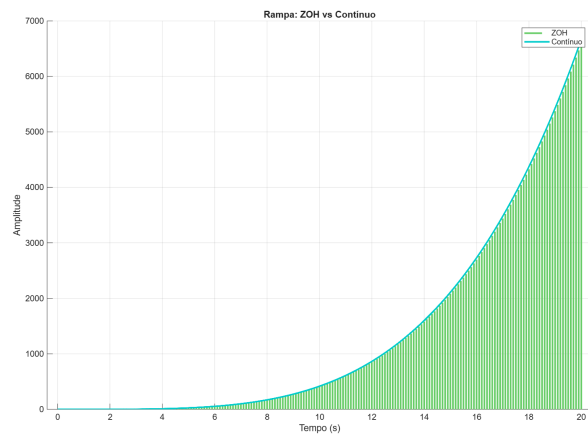


(h) Item 112:Método ZOH (Zero-Order Hold)

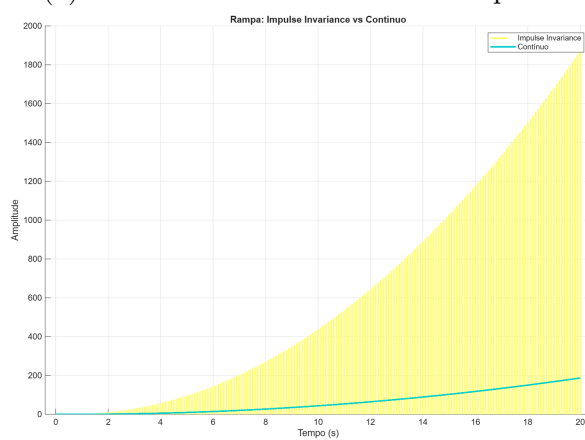
Figura 2 – Comparação das respostas ao degrau pelos dois métodos.



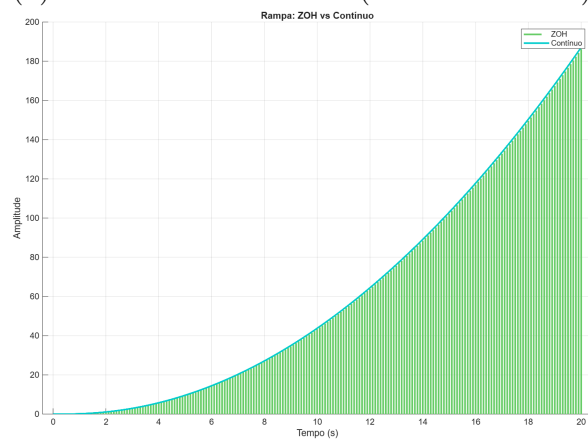
(a) Item 4:Método invariante ao impulso



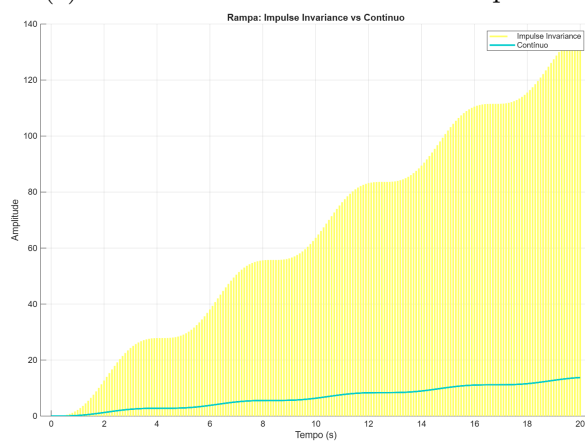
(b) Item 104:Método ZOH (Zero-Order Hold)



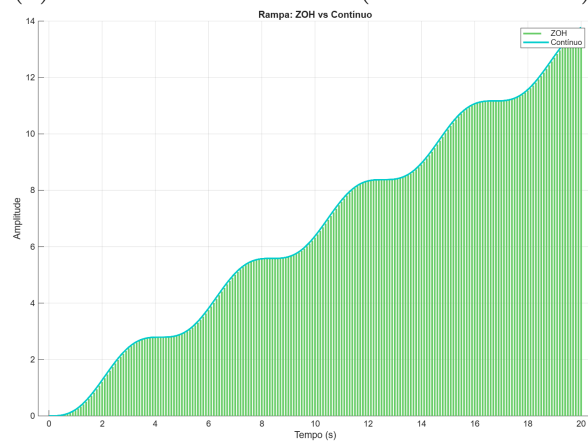
(c) Item 8:Método invariante ao impulso



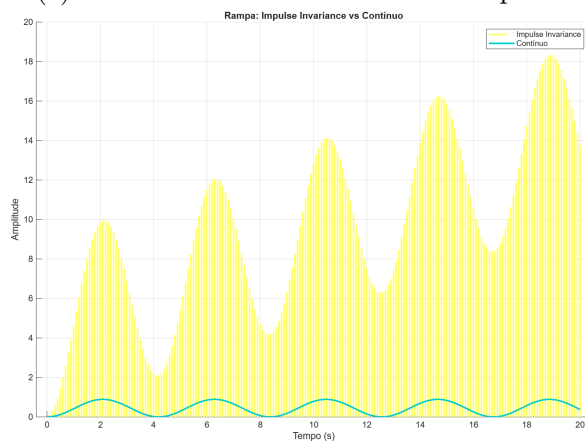
(d) Item 108:Método ZOH (Zero-Order Hold)



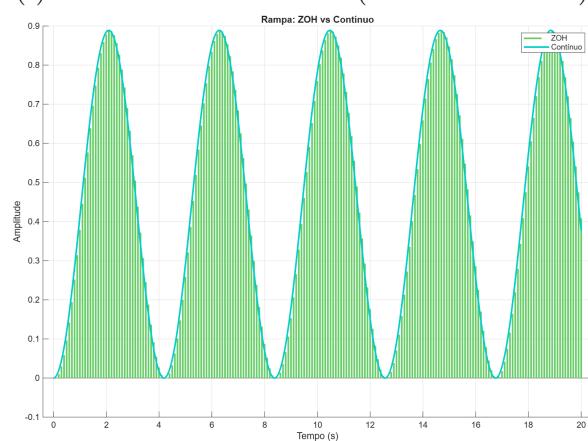
(e) Item 11:Método invariante ao impulso



(f) Item 111:Método ZOH (Zero-Order Hold)



(g) Item 12:Método invariante ao impulso



(h) Item 112:Método ZOH (Zero-Order Hold)

Figura 3 – Comparação das respostas à rampa pelos dois métodos.

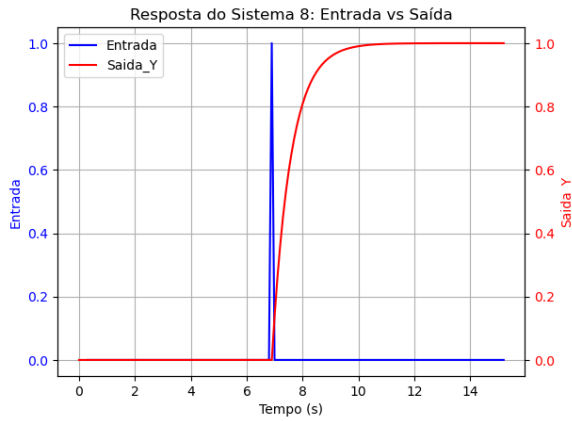
0.4.2 Implementação no ESP32

Depois de validar as equações no MATLAB, o próximo passo foi a implementação prática no microcontrolador ESP32. O código foi desenvolvido em linguagem C utilizando a IDE ESP-IDF, onde as equações de diferenças foram programadas dentro de uma tarefa periódica.

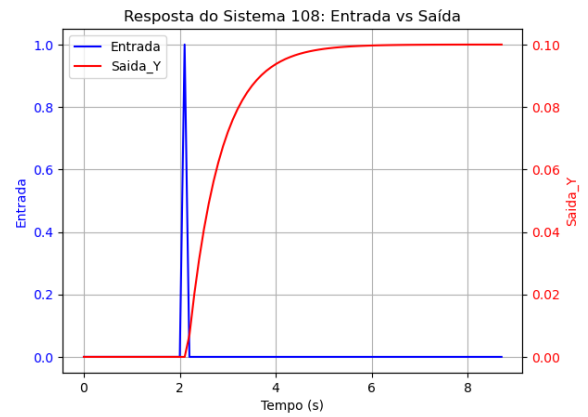
O sistema foi configurado para ser interativo via monitor serial ao enviar o número do item (exemplo: 8 ou 108), o código reseta as entradas e configura o sistema para responder de acordo com o item escolhido. Para selecionar o tipo de sinal de entrada, utiliza-se o comando 'i' para impulso, 'd' para degrau, 'r' para rampa e 'g' para zerar a entrada. Para garantir o processamento em tempo real, o sistema foi configurado para respeitar rigorosamente o tempo de amostragem $T = 0,1$ s já definido antes.

Os resultados obtidos na prática foram coletados e comparados com as curvas teóricas da simulação. As Figuras 5, 4 e 6 mostram esse comportamento no hardware. Ao analisar os gráficos, percebe-se que as características visuais das curvas (como o formato da resposta) seguiram quase exatamente o que foi visto no MATLAB.

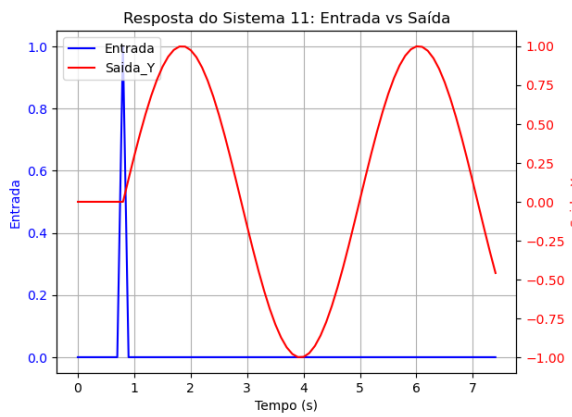
Com a análise dos gráficos obtidos no ESP32, foi possível perceber que a maior diferença entre os dois métodos de discretização foi a amplitude alcançada pelos sinais nos casos de impulso e degrau. Já para a rampa, notou-se uma diferença maior, onde o método ZOH apresentou uma aproximação melhor em relação ao modelo contínuo visto no MATLAB.



(a) Item 8:Método invariante ao impulso



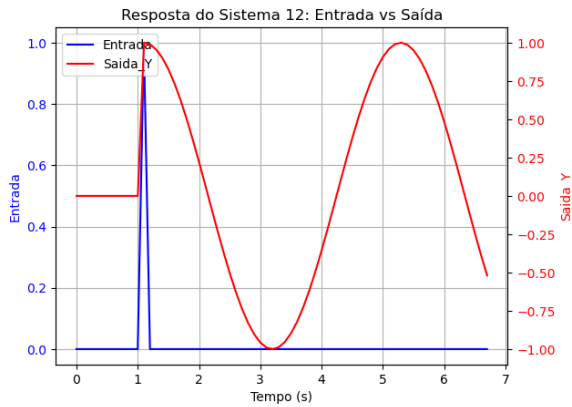
(b) Item 108:Método ZOH (Zero-Order Hold)



(c) Item 11:Método invariante ao impulso



(d) Item 111:Método ZOH (Zero-Order Hold)

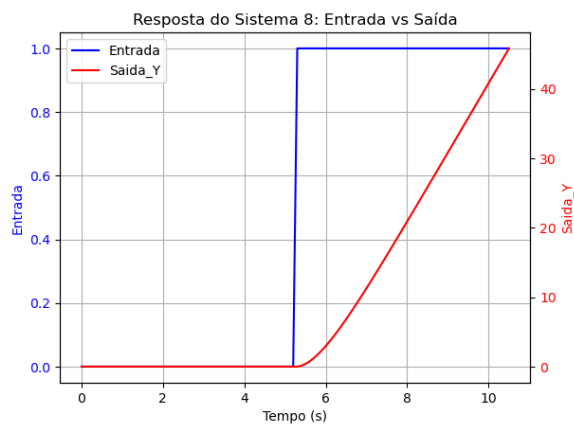


(e) Item 12: Método invariante ao impulso



(f) Item 112:Método ZOH (Zero-Order Hold)

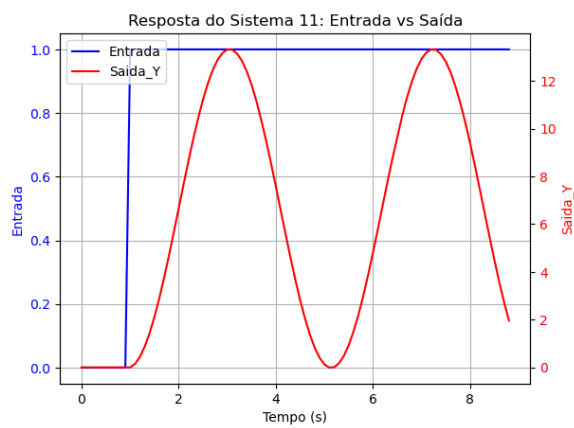
Figura 4 – implementação na ESP32, sendo a entrada um impulso.



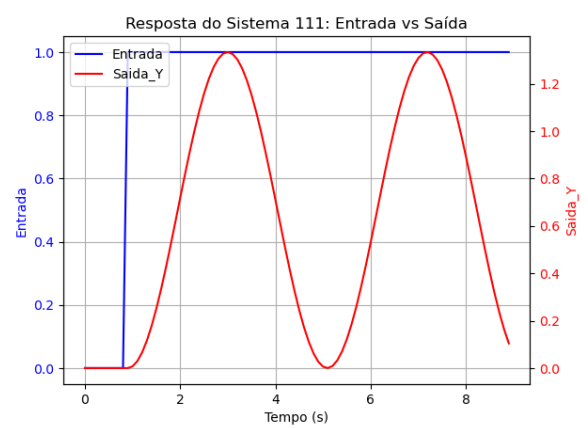
(a) Item 8:Método invariante ao impulso



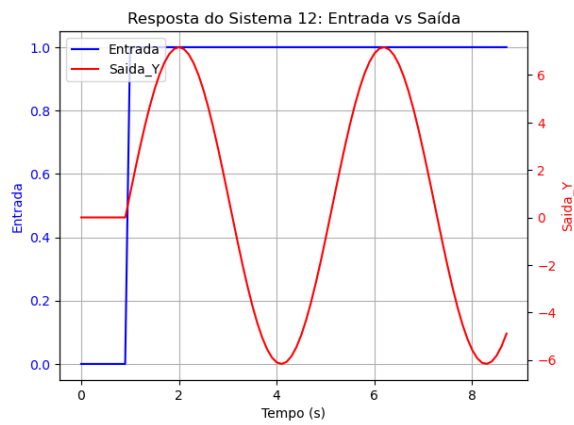
(b) Item 108:Método ZOH (Zero-Order Hold)



(c) Item 11:Método invariante ao impulso



(d) Item 111:Método ZOH (Zero-Order Hold)



(e) Item 12:Método invariante ao impulso

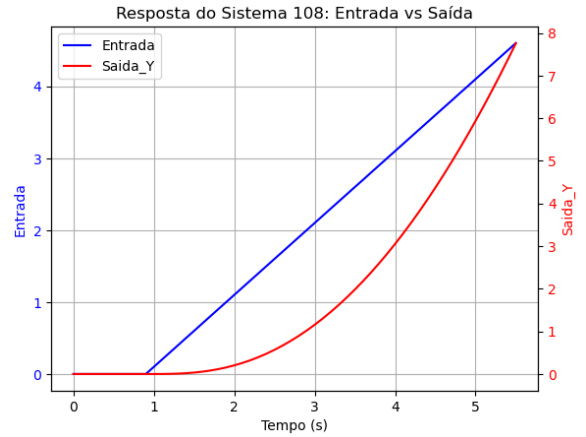


(f) Item 112:Método ZOH (Zero-Order Hold)

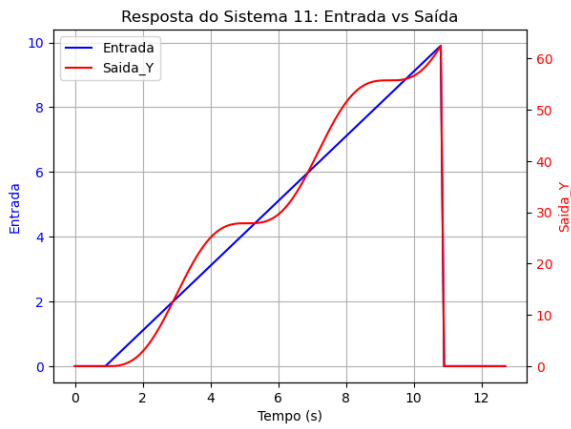
Figura 5 – Implementação no ESP32, onde a entrada é um degrau.



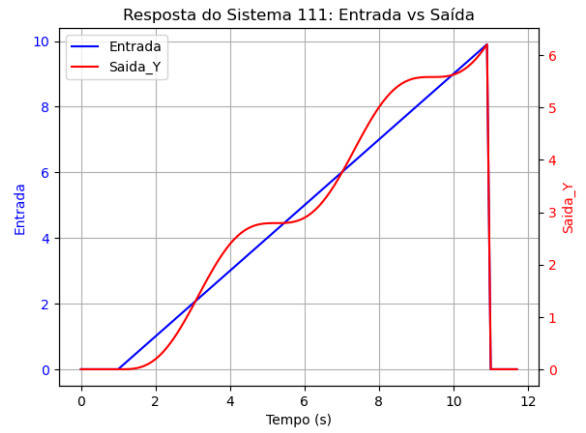
(a) Item 8:Método invariante ao impulso



(b) Item 108:Método ZOH (Zero-Order Hold)



(c) Item 11:Método invariante ao impulso



(d) Item 111:Método ZOH (Zero-Order Hold)



(e) Item 12:Método invariante ao impulso



(f) Item 112:Método ZOH (Zero-Order Hold)

Figura 6 – Comparação das respostas à rampa pelos dois métodos.

0.5 Conclusão

Este trabalho permitiu validar a implementação de sistemas de controle digital utilizando o microcontrolador ESP32. Através das simulações no MATLAB e dos testes práticos no hardware, foi possível observar como diferentes métodos de discretização se

comportam diante de sinais de entrada variados.

A análise dos resultados mostrou que não existe um método único que seja superior em todas as situações. Enquanto a Invariância ao Impulso apresentou bons resultados para entradas de impulso, o método ZOH (Zero-Order Hold) mostrou-se mais eficiente e preciso para entradas do tipo degrau e rampa, mantendo uma rastreabilidade mais fiel ao modelo contínuo original.

Referências

- [1] AGUIRRE, L. A. **Controle de Sistemas Amostrados**. 1. ed. São Paulo: Blucher, 2019.
- [2] ESPRESSIF SYSTEMS. **ESP-IDF Programming Guide**. Disponível em: <<https://docs.espressif.com/projects/esp-idf/>>. Acesso em: 21 abr. 2026.
- [3] Luan Italo. **AV1: Implementação de Controle Digital no ESP32**. GitHub, 2026. Disponível em: <https://github.com/luanjjbr/AV1.git>. Acesso em: 21 abr. 2026.
- [4] Luan Italo. **AV1: Implementação de Controle Digital no ESP32**. GitHub, 2026. Disponível em: <https://github.com/luanjjbr/AV1/tree/main/Matlab>. Acesso em: 21 abr. 2026.